

软件安全实验五

班级：2022211801 学号：2022211570 姓名：项枫

一、目标

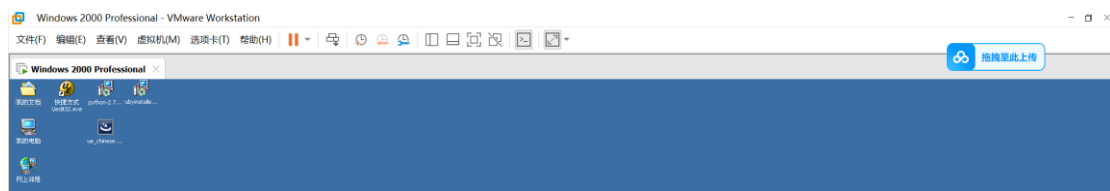
- 1、了解 SEH 攻击及虚函数攻击的基本原理。
- 2、通过调试 SEH 攻击代码，理解 Windows 异常处理机制，掌握针对 SEH 的攻击方式，并利用 OllyDbg 跟踪异常状态。
- 3、调试虚函数攻击代码，理解虚函数工作机制与内存分布方式，掌握基本的虚函数攻击与计算方式，并可以用 OllyDbg 追踪。

二、测试步骤与结果

(一) S.E.H 实验

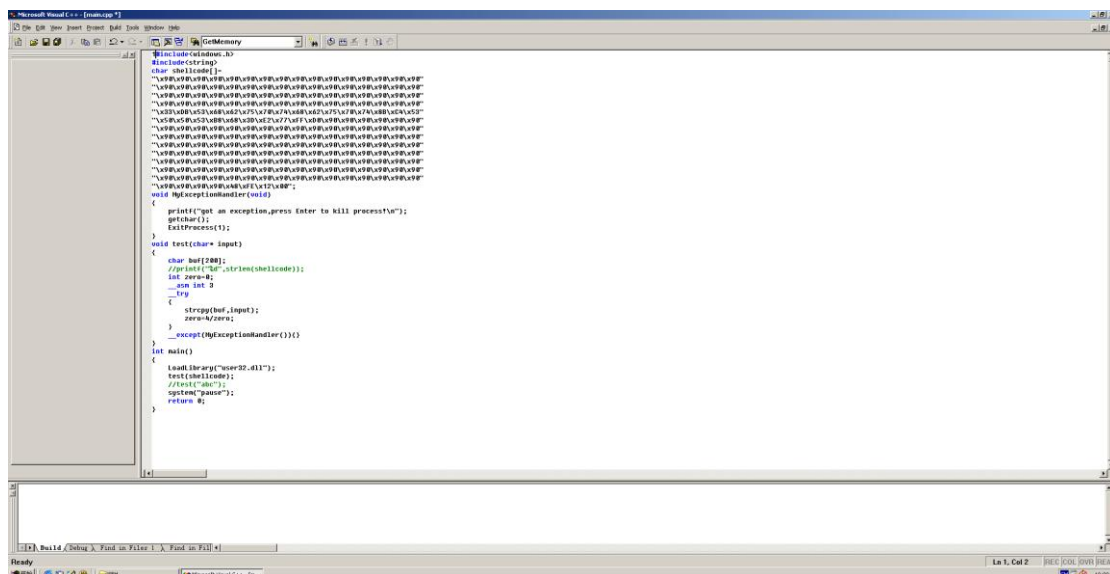
1、进入虚拟机

打开 VM workstation，点击文件->打开->选择我们要使用的 WIN2000 的 VMX 文件，之后我们就进入了虚拟机。



2、研究代码

使用桌面的 VC6 打开文件夹“实验题目/SEH”中的 main.cpp，对代码进行研究。



通过分析上图所示代码，结合所学内容，可以获取以下信息：

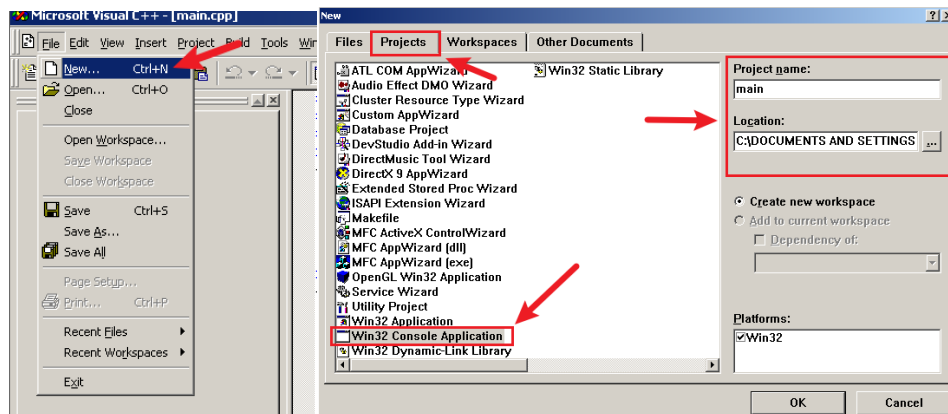
(1) 函数 `MyExceptionHandler()` 是异常处理函数；

(2) `test()` 函数的 `strcpy` 处是典型栈溢出漏洞；

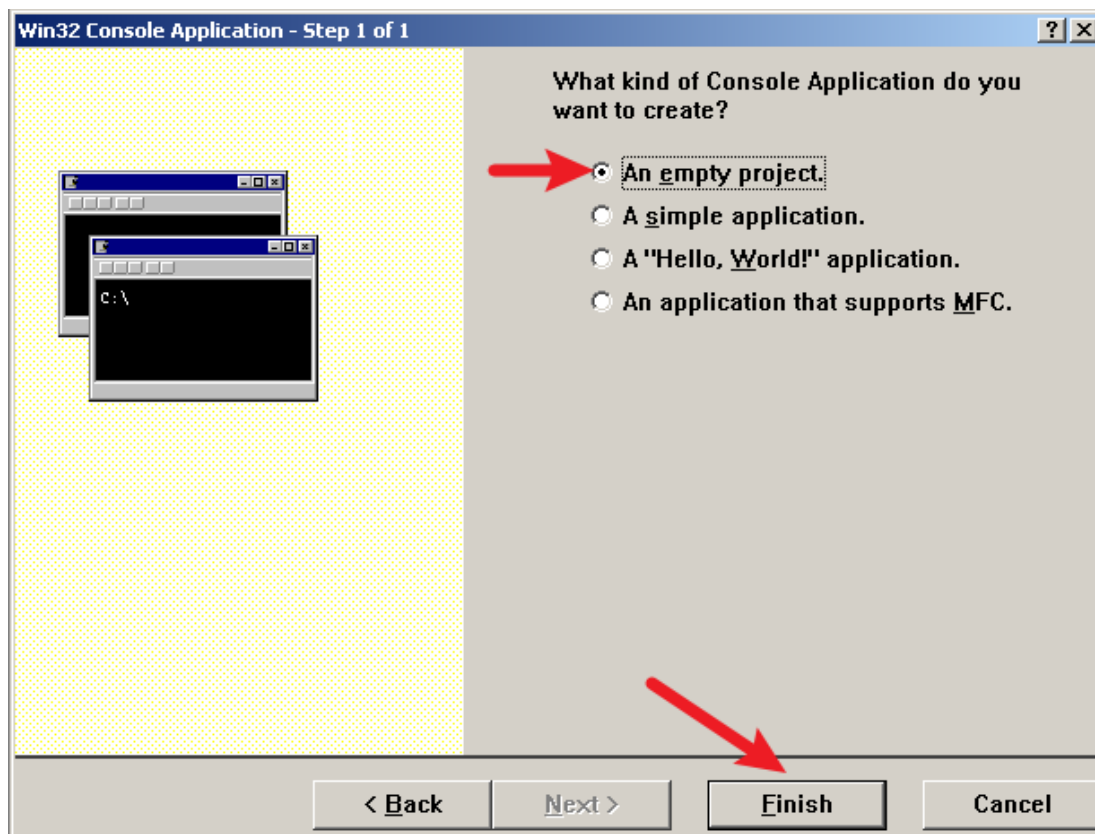
(3) `_try{}{}` 在 `test` 函数栈帧中安装一个 S.E.H 结构，其中除 0 操作会产生异常。`strcpy` 操作没有产生溢出时，除 0 操作产生的异常会被异常处理函数处理，而当 `strcpy` 操作产生溢出时，会将栈帧中 S.E.H 异常处理句柄改为 shellcode 入口地址，代码植入成功。

3、生成工程

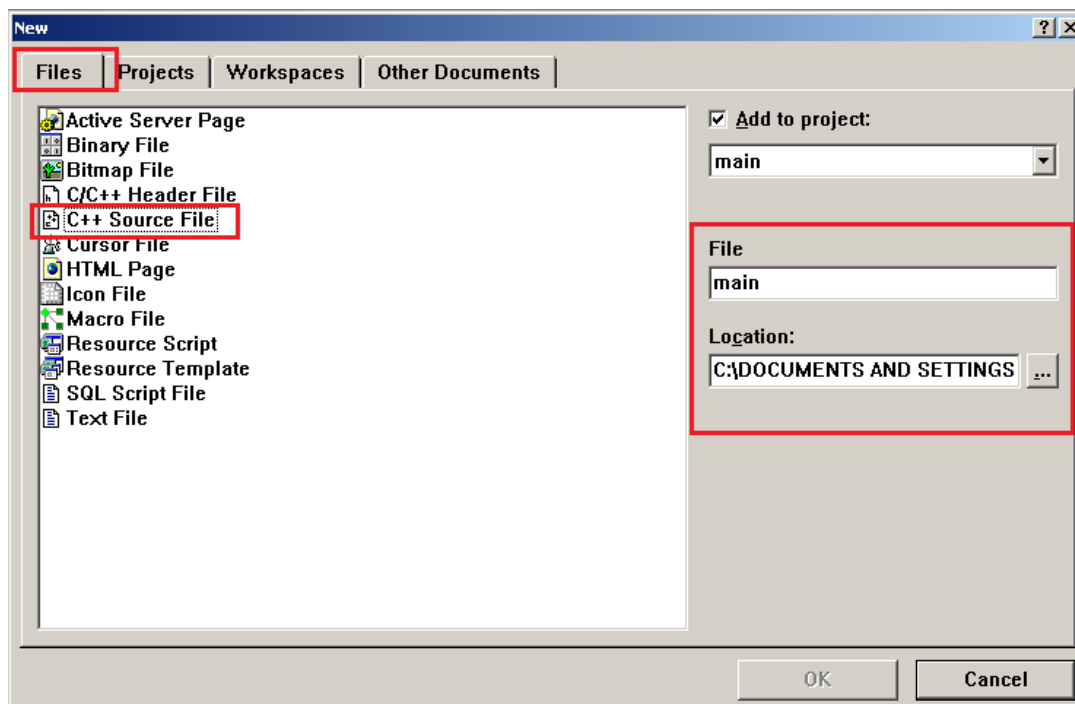
(1) 在 VC 中，点击 `File->new->Projects->Win32 Console Application`，右上角设置好 project 的名称和存放路径。



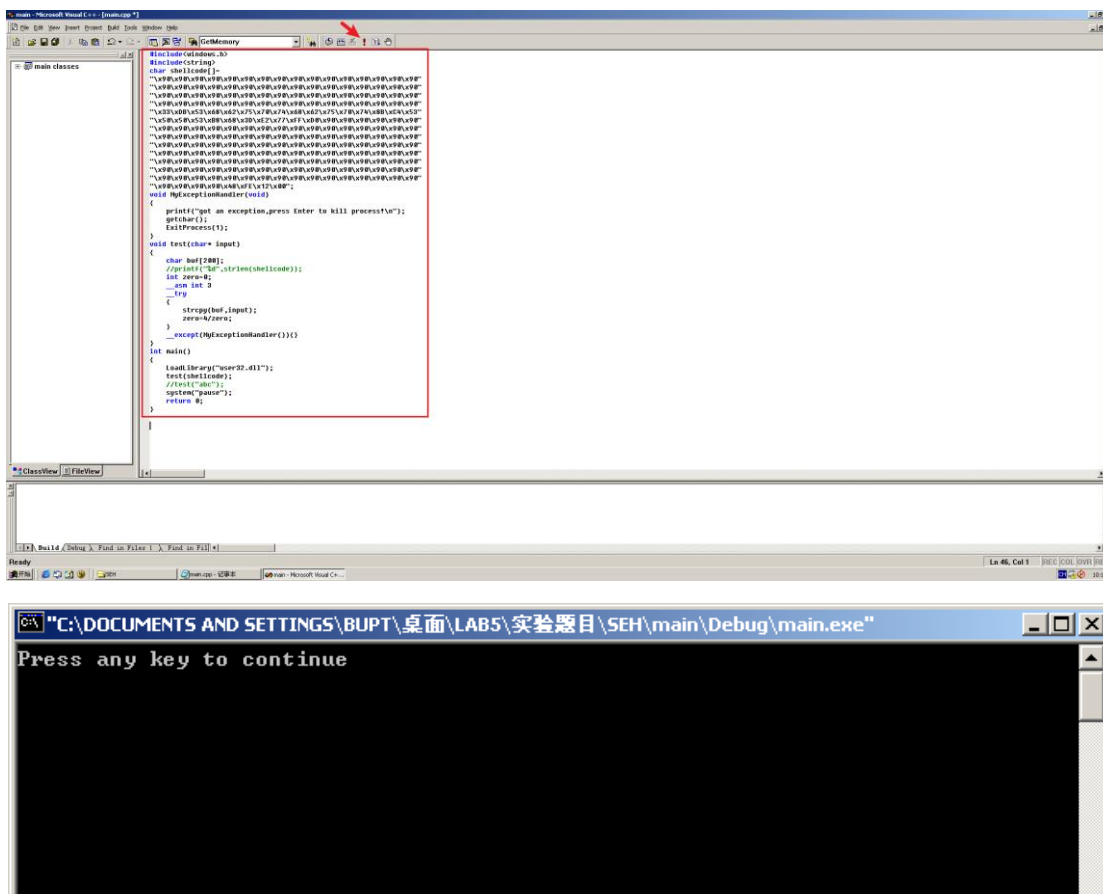
(2) 点击 `OK->An empty project->Finish`



(3) 之后再 new->Files->C++ Source File, 右侧设定好添加到的项目(我们刚才新建的项目), 文件名和路径。



(5) 完成新建文件后, 在新建的文件下编写或复制刚才我们看到的 main 程序, 然后点击右上的红色感叹号执行。

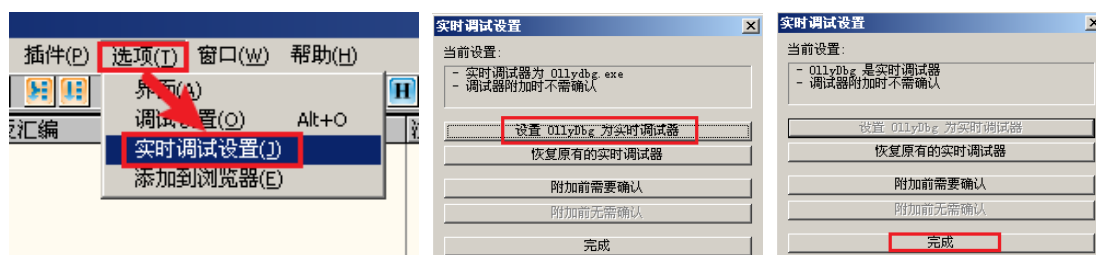


4、运行 ollydbg

桌面找到快捷方式 ollydbg.exe，双击进入。

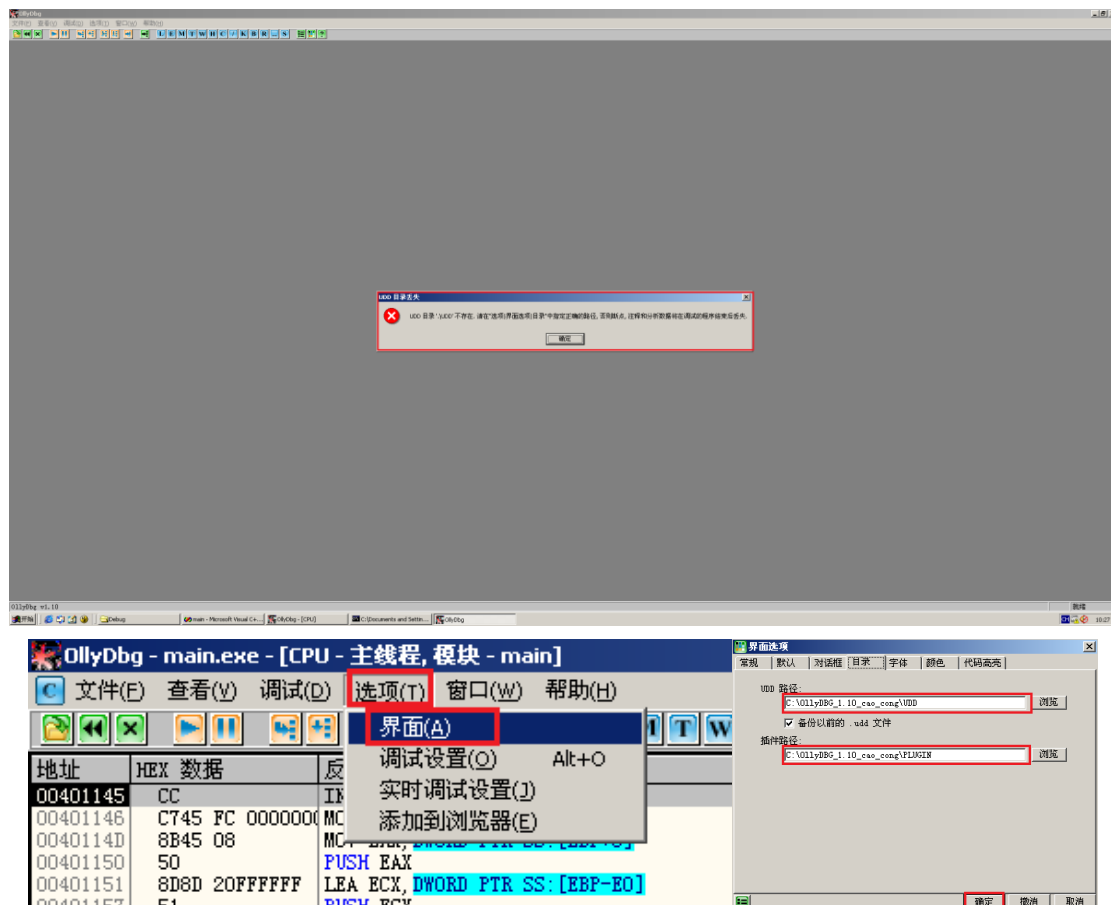


为了能触发 int 3 断点时启动 OllyDbg，我们选择选项中的实时调试设置选择设置 OllyDbg 为实时调试器，然后当我们运行 exe 文件时，int 3 断点触发后就会启动 OllyDbg。

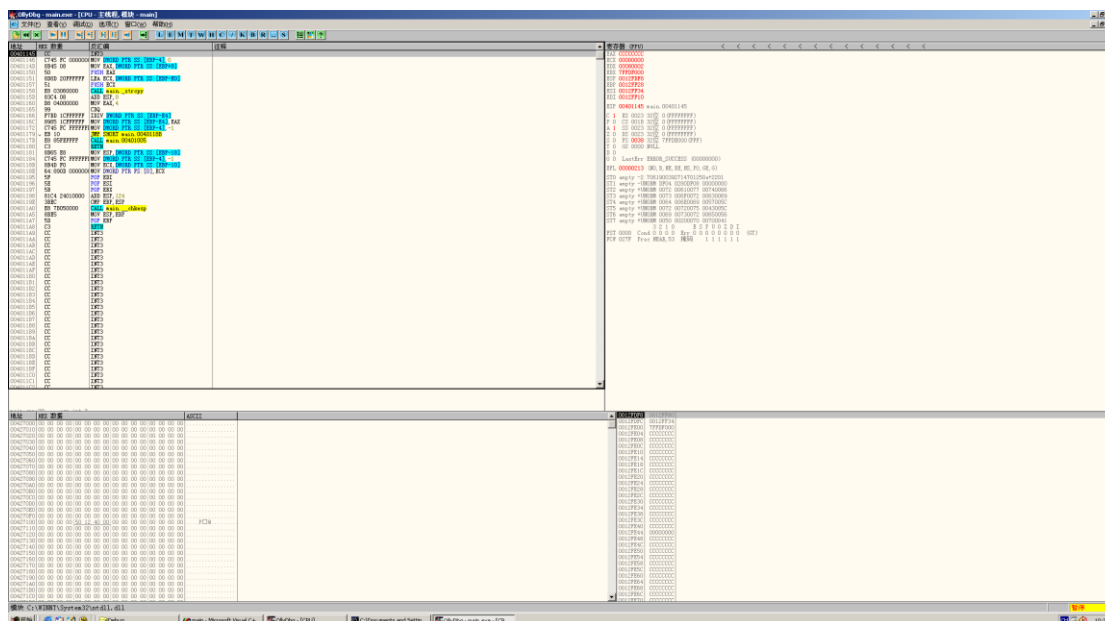


5、在 int 3 上启动 OllyDbg

运行刚刚创建的 main.exe，发现要求创建 UDD 目录。

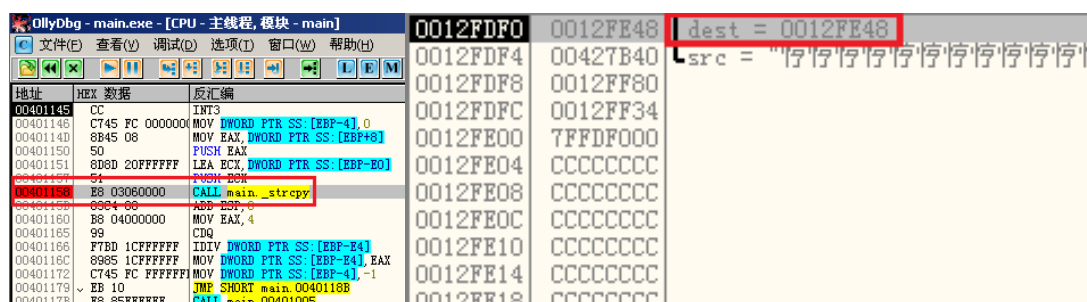


两个路径设置成功后，重新运行 main.exe 程序，成功在 int 3 上启动 OllyDbg。

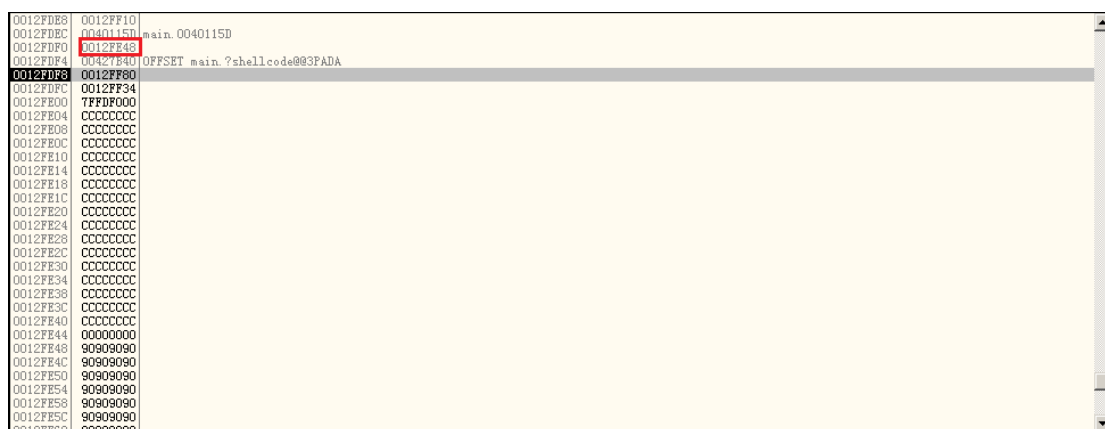


6、确定 shellcode 的起始位置

在 strcpy 函数处设置断点，程序运行到此处时观察右下角缓冲区数据，可以看到在执行 strcpy 函数之前，shellcode 的起始地址为 0x0012FE48。

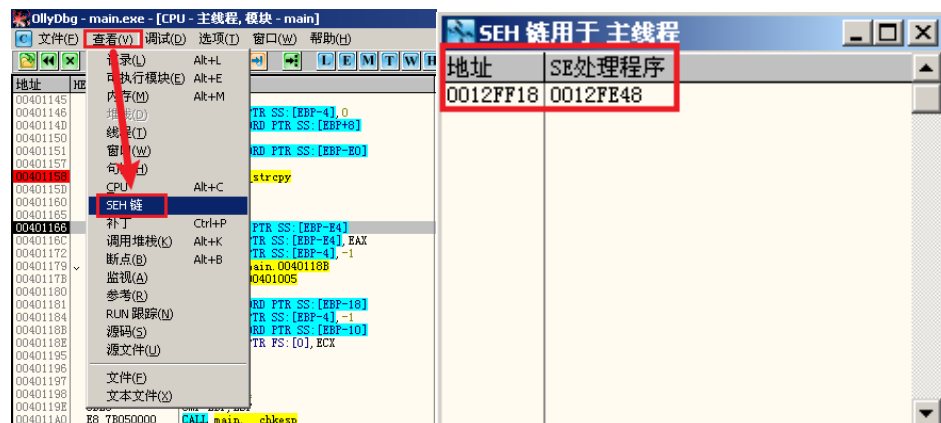


执行完 strcpy，确认 shellcode 的起始位置是 0x0012FE48。



7、查看 S.E.H 链

点击查看菜单中的 SEH 链，可以看到 S.E.H 链的情况。可以看到地址为 0x0012FF18。

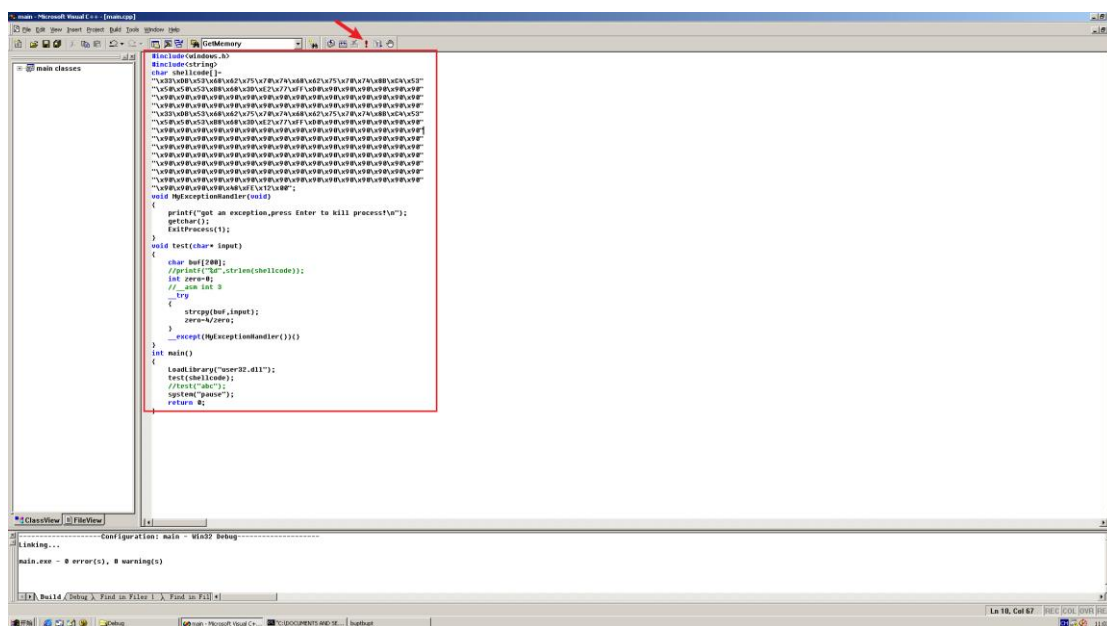


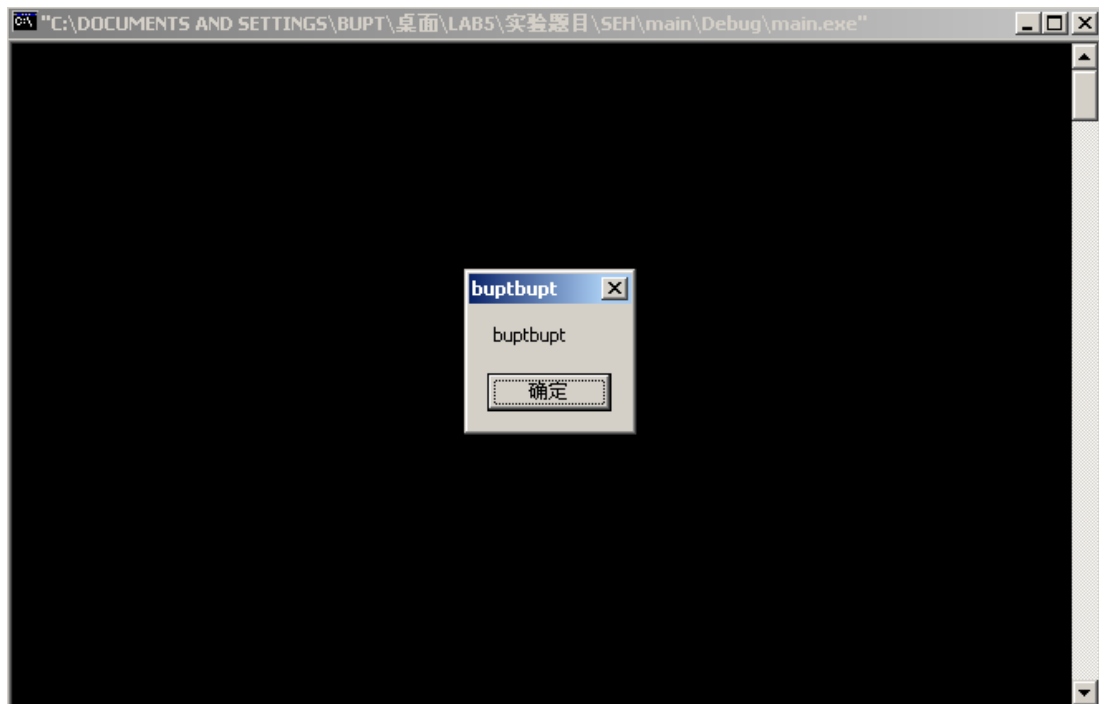
查看地址 0x0012FF18 的记录，发现其指向下一个 SEH 指针，接着是异常处理程序，只需要把 0x0012FF1C 这个地址的内容改成 shellcode 起始地址即可。

0012FF14	90909090	
0012FF18	90909090	指向下一个 SEH 记录的指针
0012FF1C	0012FE48	SE 处理程序
0012FF20	00425058	main.00425058
0012FF24	00000000	
0012FF28	0012FF80	
0012FF2C	00401216	返回到 main.00401216 来自 main.00401014
0012FF30	00427B40	OFFSET main.?shellcode@@@3PADA
0012FF34	80020003	
0012FF38	0280DCA0	
0012FF3C	7F8F8000	

8、异常处理程序

由于 shellcode 的起始地址为 0x0012FE48，第一个 S.E.H 地址为 0x0012FF18(指向下一个 S.E.H 的指针) 0x0012FF1C(异常处理地址)，因此 shellcode 需要使用 0x0012FF1C-0x0012FE48=212 个字节进行填充，且注释掉 __asm int 3。修改后的程序如下，然后点击右上的红色感叹号执行。

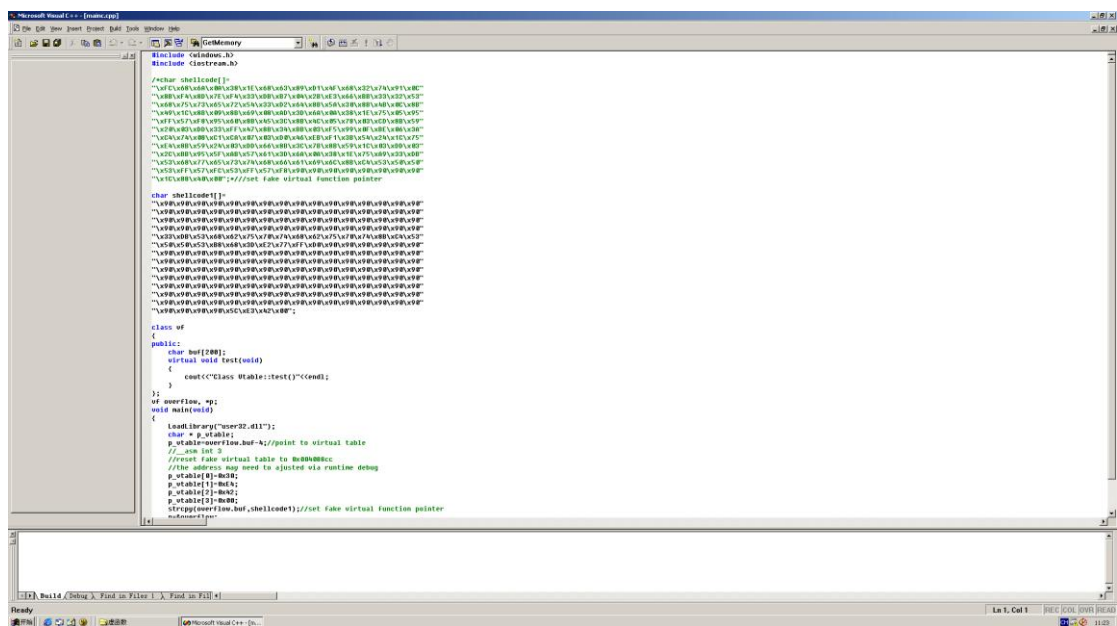




(二) 虚函数攻击

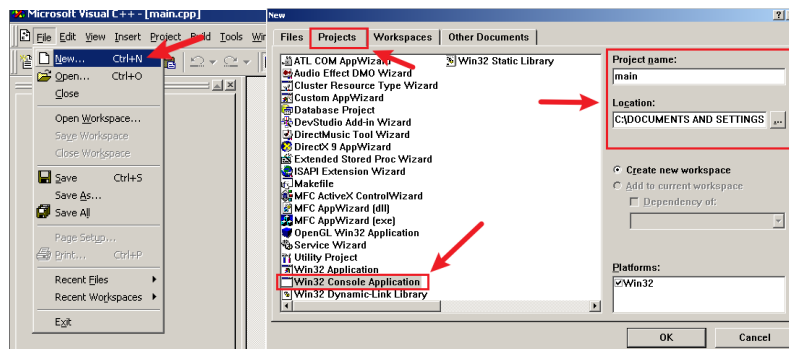
1、研究代码

使用桌面的 VC6 打开文件夹“实验题目/虚函数”中的 mainc.cpp，对代码进行研究。

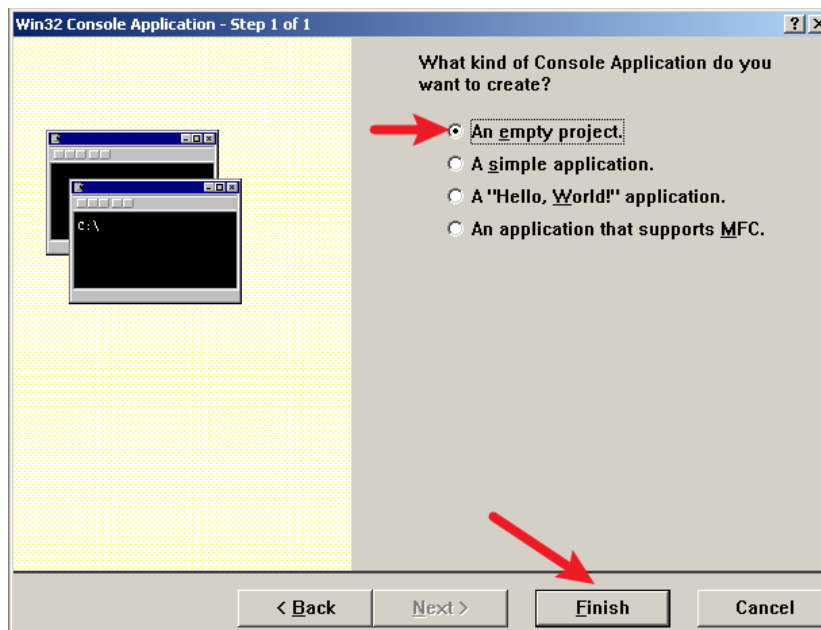


2、生成工程

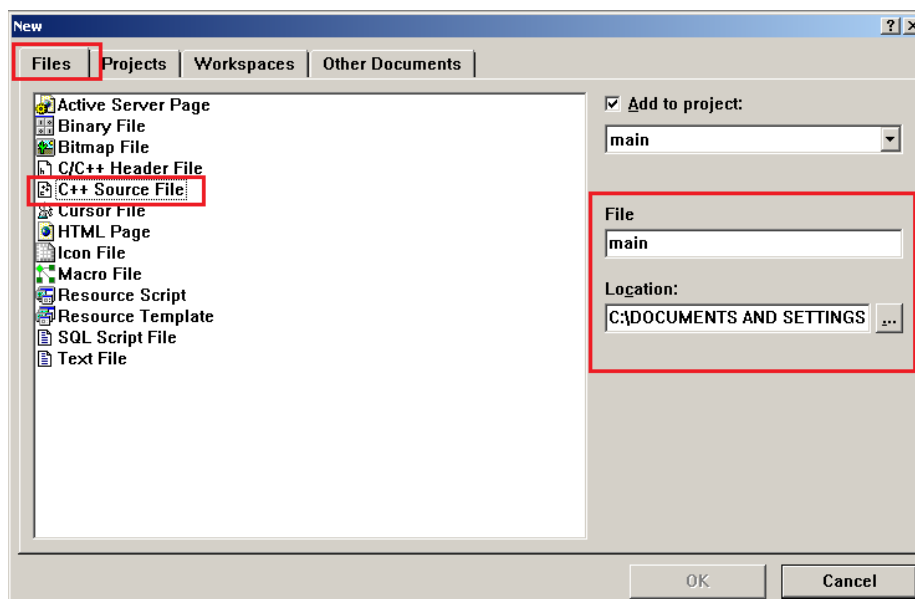
(1) 在 VC 中，点击 File->new->Projects->Win32 Console Application，右上角设置好 project 的名称和存放路径。



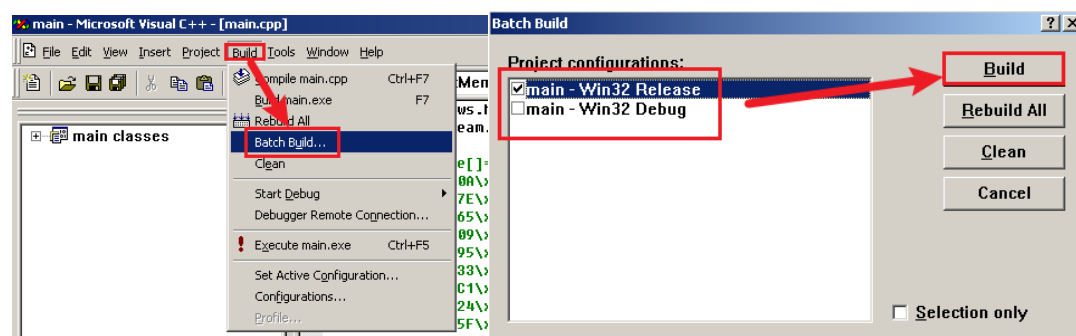
(2) 点击 OK->An empty project->Finish



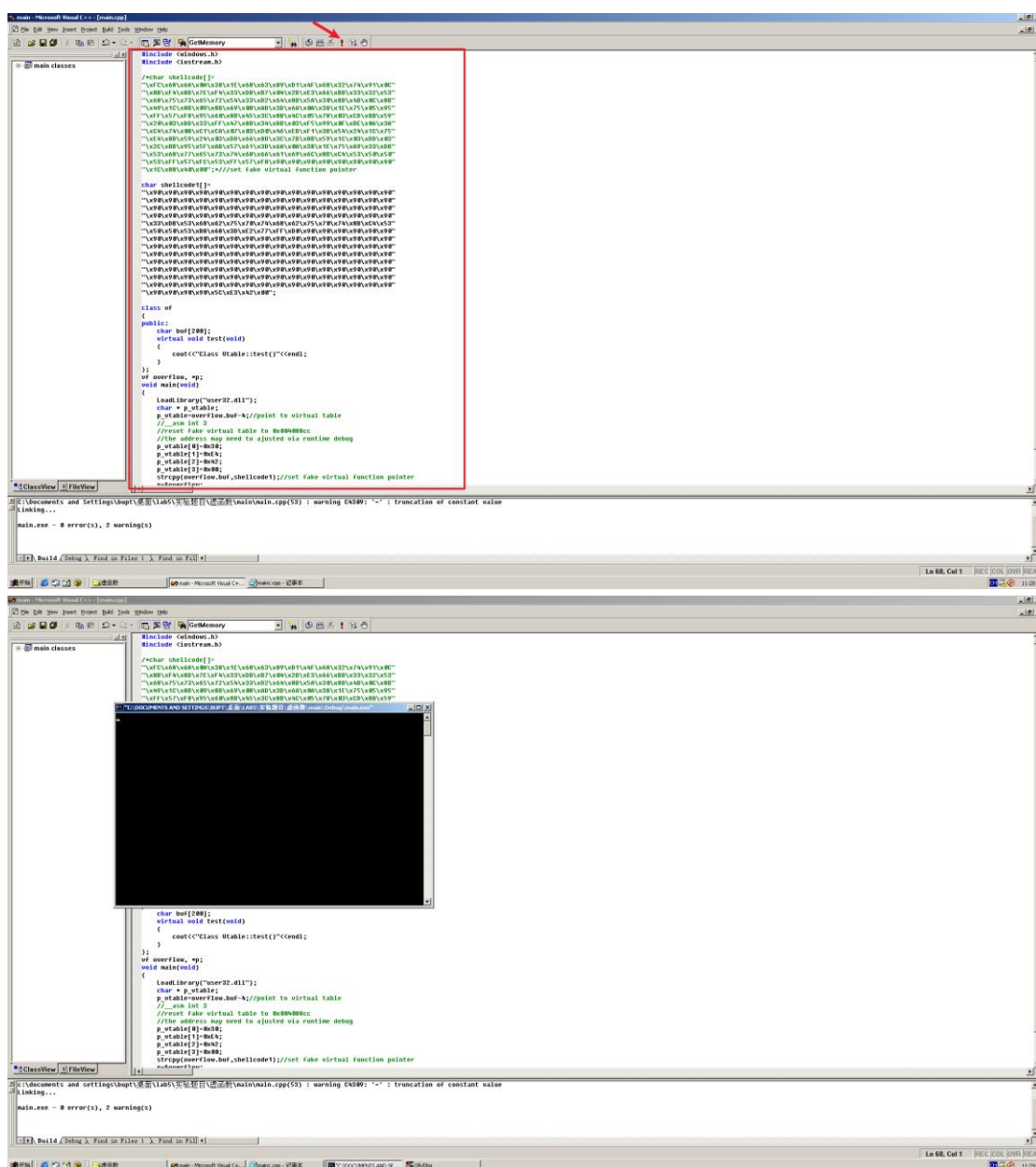
(3) 之后再 new->Files->C++ Source File, 右侧设定好添加到的项目(我们刚才新建的项目), 文件名和路径。



(5) 设置 VC6 的 build 版本为 release。

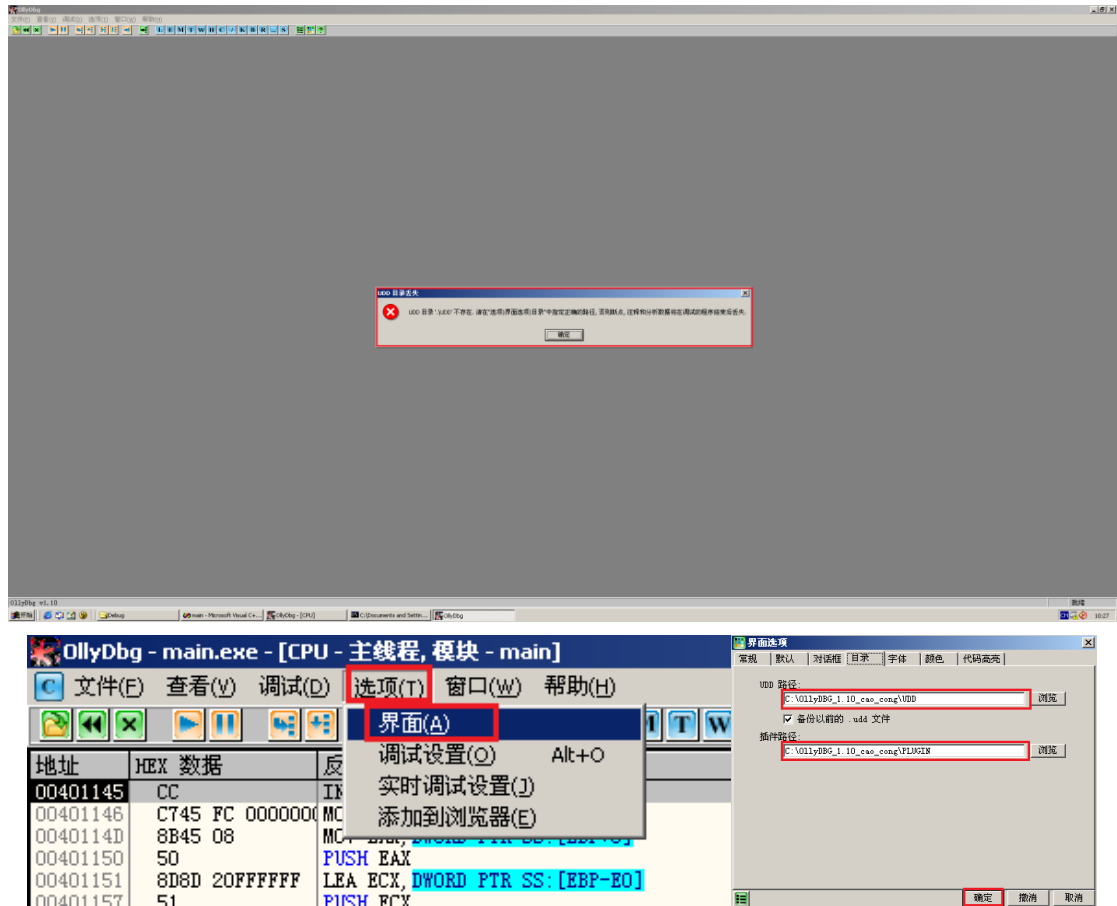


(6) 完成新建文件后，在新建的文件下编写或复制刚才我们看到的 main 程序，然后点击右上的红色感叹号执行。

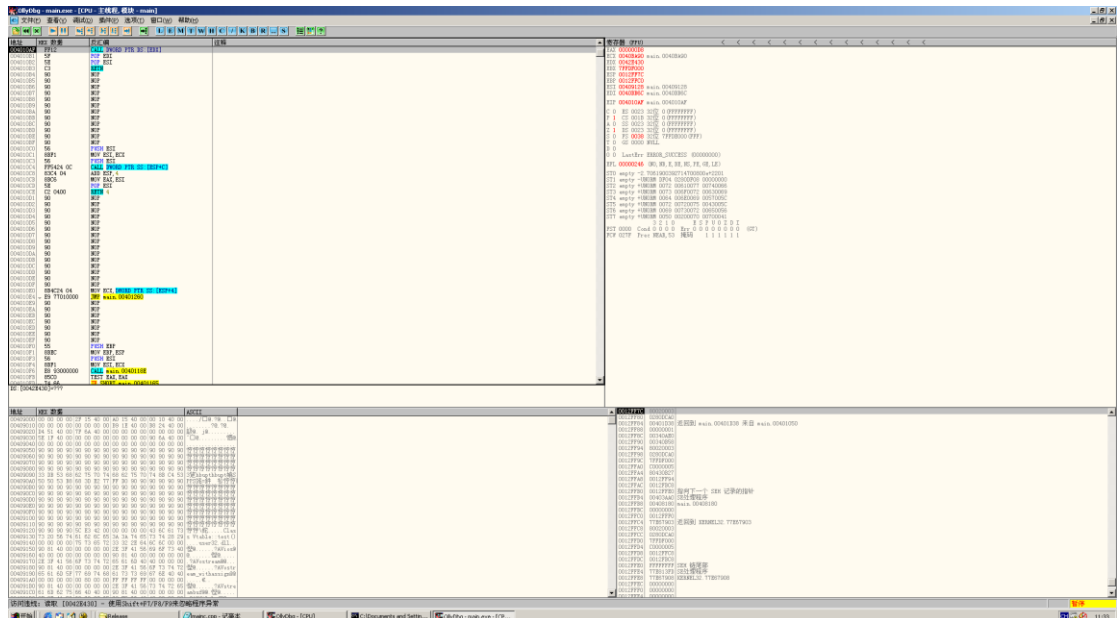


2、在 int 3 上启动 OllyDbg

运行刚刚创建的 main.exe，发现要求创建 UDD 目录。



两个路径设置成功后，重新运行 main.exe 程序，成功在 int 3 上启动 OllyDbg。



3、查看 shellcode 的起始地址

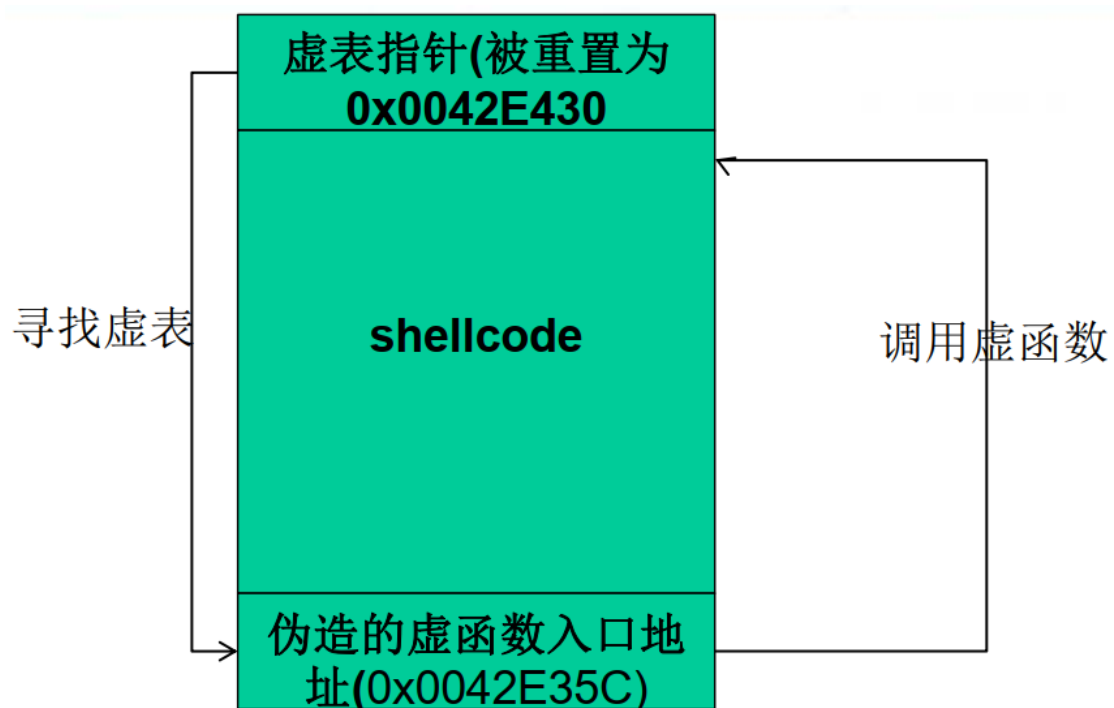
在 strcpy 函数处设置断点，程序运行到此处时观察右下角缓冲区数据，可以看到在执行 strcpy 函数之前，shellcode 的起始地址为 0x0042E27C。

三、测试结论

通过本次实验，对 SEH 的初步理解，S.E.H 结构体存放在系统栈中。

如果程序源代码中使用了 `__try {}__except {}` 或者 `Assert` 宏等异常处理机制，编译器将最终通过向当前函数栈帧中安装一个 S.E.H 来实现异常处理。栈中的多个 S.E.H 通过链表指针在栈内由栈顶向栈底串成单向链表，位于链表最顶端的 S.E.H 通过 T.E.B（线程环境块）0 字节偏移处的指针标识，FS 寄存器指向 TEB 的位置。当异常发生时，操作系统会中断程序，并首先从 T.E.B 的 0 字节偏移处取出距离栈顶最近的 S.E.H，使用异常处理函数句柄所指向的代码来处理异常。因此 S.E.H 存放在栈内，溢出缓冲区的数据有可能淹没 S.E.H。精心制造的溢出数据可以把 S.E.H 中异常处理函数的入口地址更改为 shellcode 的起始地址。当 Windows 开始处理溢出后的异常时，会错误地把 shellcode 当做异常处理函数而执行，从而达到攻击效果。

对于虚函数攻击，一般在一个类中一旦有多态产生就会有虚函数表存在，该表一般位于类的成员变量上方。当实例化一个类时，成员函数与静态成员都不算在类对象内部。只算成员变量的大小，所以也可以那么认为：存储类实例化对象上方就是虚函数表的地址。在这种情况下，假设类中有栈内有分配了空间，我们用指针就可以寻找到该空间的上面 4 字节（32 位）就可以修改对应的虚表指针转到自己的 shellcode 上了，在调用了对应的虚表指针指向的虚函数，就相当于调用了自己的 shellcode 了。虚函数攻击原理如下：

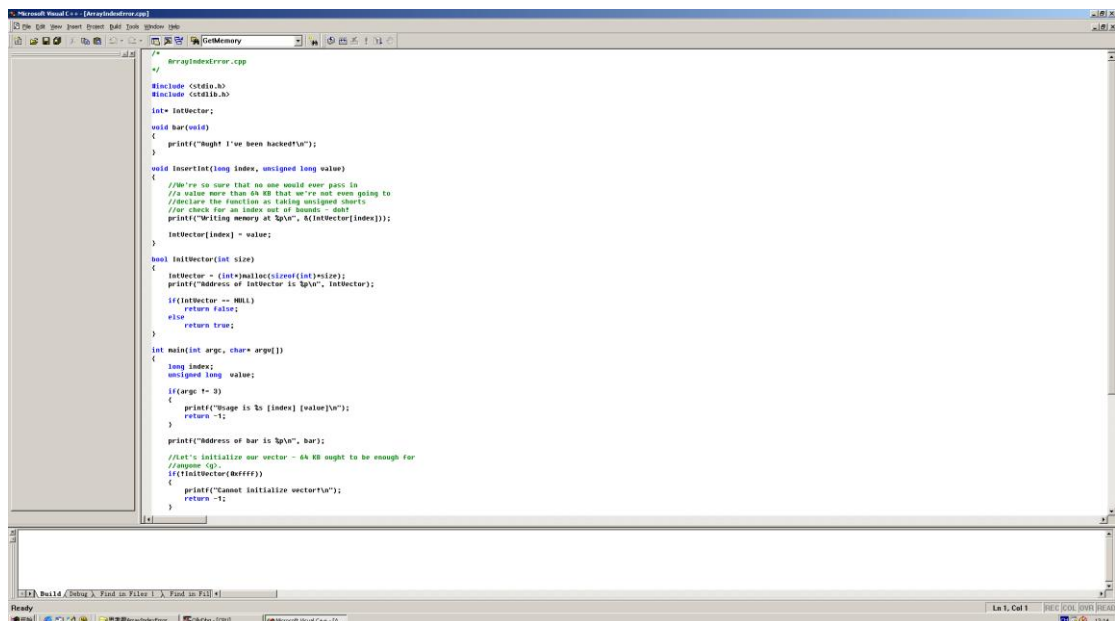


四、思考题

针对 Arrayindexerror 数组索引思考题程序，在不修改源代码的情况下，研究如何攻击目标代码，并调用 bar 函数。

1、查看源码

打开“思考题 ArrayIndexError/ArrayIndexError.cpp”，通过分析可得，源代码中存在 ArrayIndex error，即数组下标错误。



```
/*
 * ArrayIndexError.cpp
 */
#include <stdio.h>
#include <stdlib.h>

int* IntVector;

void bar(void)
{
    printf("Hight I've been hacked!\n");
}

void InsertInt(long index, unsigned long value)
{
    //We're so sure that no one would ever pass in
    //a value more than 64 KB that we're not even going to
    //declare the function as taking unsigned shorts
    //or check for an index out of bounds - duh!
    printf("Writing memory at %p\n", &IntVector[index]);
    IntVector[index] = value;
}

bool InitVector(int size)
{
    IntVector = (int*)malloc(sizeof(int)*size);
    printf("Address of IntVector is %p\n", IntVector);
    if(IntVector == NULL)
        return false;
    else
        return true;
}

int main(int argc, char* argv[])
{
    long index;
    unsigned long value;
    if(argc != 3)
    {
        printf("Usage is %s %s %s\n", argv[0], argv[1], argv[2]);
        return -1;
    }

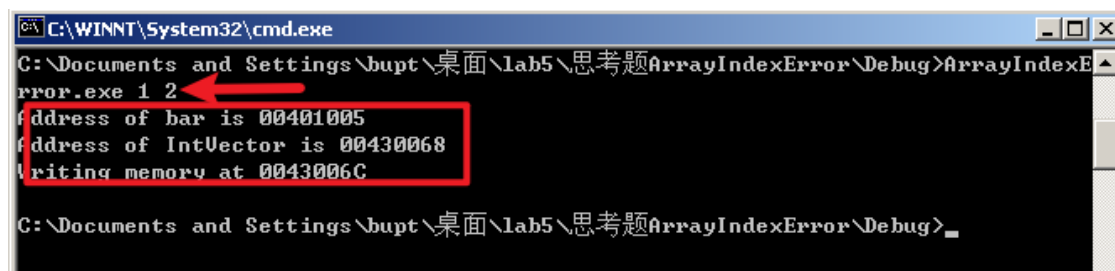
    printf("Address of bar is %p\n", bar);
    //Let's initialize our vector - 64 KB ought to be enough for
    //anyone - QP.
    if(!InitVector(65536))
    {
        printf("Cannot initialize vector!\n");
        return -1;
    }
}
```

2、执行程序

在命令行执行程序，尝试输入参数 1 2。

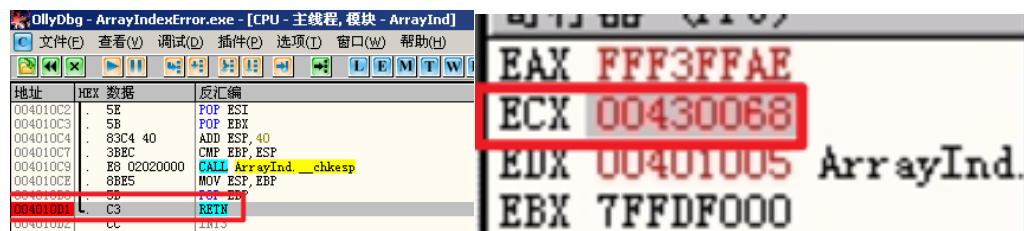
可以看到，bar 函数的地址为 0x00401005，数组是从 0x00430068 开始，写入的数值的位置为 0x0043006C。（该值随输入的参数的值的变化而变化，例如我输入的第一个参数是 1，则此时是在 0x0043006C 写入数值 2）。

进一步分析，若要实现 bar 函数的调用，在命令行输入的 2 个参数就十分关键。通过研究代码逻辑，可以发现，在调用 InsertInt(index, value)后，将返回 main 函数。那么只需要找到这个过程中保存栈的返回值的地址，并将返回值修改为 bar 函数的地址，并根据这两项信息确定 2 个参数的值，再运行程序，就可以调用 bar 函数了。



```
C:\WINNT\System32\cmd.exe
C:\Documents and Settings\bupt\桌面\lab5\思考题ArrayIndexError\Debug>ArrayIndexError.exe 1 2
Address of bar is 00401005
Address of IntVector is 00430068
Writing memory at 0043006C
C:\Documents and Settings\bupt\桌面\lab5\思考题ArrayIndexError\Debug>
```

3、返回地址位置打断点，查看返回指针位置，找到返回地址为 0x00430068。



4、回溯到 bar 地址

$0x0012FFC4 - 0x00430068 = 0xFFCFFF5C$

0012FFB4	77E67900	KERNEL32.77E67900
0012FFB8	FFFFFFFF	
0012FFEC	00000000	
0012FFED	0012FFED	
0012FFC4	77E67903	返回到 KERNEL32.77E67903
0012FFC8	00000000	
0012FFCC	00000000	
0012FFD0	7FFDF000	
0012FFD4	00000000	
0012FFD8	0012FFC8	

5、计算参数

第一个参数：由于输入信息为 int 型，因此计算是按照四倍计算，需要除 4 得到结果，即 $0xFFCFFF5C / 4 = -786514$ 。

第二个参数：bar 的地址 10 进制 $0x00401005 = 4198405$ 。

因此传入参数 -786514 4198405。

6、再次运行程序

在命令行执行程序，输入参数 -786514 4198405，成功！

